

Series XIX – Article XIX.2: Boolean Circuit for Tetrahedral Sum Testing

Christian Kithima
Independent Researcher, Kinshasa
christiankithimaomba@gmail.com
WhatsApp: +243995176701
Telegram: +243818136082

May 2026

Abstract

We construct a boolean circuit that, for a fixed positive integer N with $1 \leq N \leq N_0$ (where N_0 is the effective bound from Article XIX.1), decides whether N can be expressed as a sum of at most five tetrahedral numbers $T_a = a(a+1)(a+2)/6$. The circuit takes as input the binary representations of five non-negative integers a, b, c, d, e (each bounded by $\lceil \sqrt[3]{6N} \rceil$) and outputs 1 if and only if $T_a + T_b + T_c + T_d + T_e = N$. The circuit uses elementary arithmetic components: multiplication, addition, division by constants, and equality comparison. Its size is polynomial in $\log N$. Using the Tseitin transformation, we convert this circuit into a 3-SAT instance Φ_N that is satisfiable iff a representation exists. The SAT instance has $M = O((\log N)^2 \log \log N)$ variables and is the input to the spectral Hamiltonian in Article XIX.3. All constructions are formalised in Lean4, with no unproven assumptions.

Contents

1	Introduction	1
2	Preliminaries	2
3	Circuit for a tetrahedral number	2
4	Summation and comparison	2
5	Reduction to 3-SAT via Tseitin	3
6	Formalisation in Lean4	3
7	Conclusion	4

1 Introduction

Article XIX.1 established that every sufficiently large integer $N > N_0$ can be expressed as a sum of five tetrahedral numbers. For the remaining finite range $1 \leq N \leq N_0$ we must verify the existence of a representation. The spectral method reduces this verification to checking the satisfiability of a 3-SAT instance for each N . In this article we construct the boolean circuit that tests the tetrahedral sum condition for a given N and for candidate indices a, b, c, d, e .

2 Preliminaries

Fix a positive integer N with $N \leq N_0$. Let

$$M = \lceil \sqrt[3]{6N} \rceil.$$

All indices with $T_a \leq N$ satisfy $a \leq M$. We represent each index by $L = \lceil \log_2(M + 1) \rceil$ bits. The total number of input bits is $5L$.

We assume the existence of polynomial-size circuits for:

- Multiplication: $\text{MUL}(x, y)$ produces a $2L$ -bit product.
- Addition of a constant: $\text{ADD_CONST}(x, c)$ (e.g., $x + 1$, $x + 2$).
- Equality comparison: $\text{EQ}(x, y)$ outputs 1 iff $x = y$.
- Division by a constant: $\text{DIV_CONST}(x, 6)$ (exact division).

3 Circuit for a tetrahedral number

We need a circuit that, given an L -bit number x , computes $T_x = x(x + 1)(x + 2)/6$. The computation proceeds as follows:

1. Compute $x_1 = \text{ADD_CONST}(x, 1)$.
2. Compute $x_2 = \text{ADD_CONST}(x, 2)$.
3. Compute $p_1 = \text{MUL}(x, x_1)$.
4. Compute $p_2 = \text{MUL}(p_1, x_2)$.
5. Compute $t = \text{DIV_CONST}(p_2, 6)$.

The size of each multiplication is $O(L^2)$; addition and division by a constant are $O(L)$. Thus the tetrahedral circuit has size $O(L^2)$.

4 Summation and comparison

We have five inputs a, b, c, d, e . For each, we compute T_a, T_b, T_c, T_d, T_e using five copies of the tetrahedral circuit. Then we compute the total sum

$$S = T_a + T_b + T_c + T_d + T_e$$

using a binary adder tree (depth $\lceil \log_2 5 \rceil = 3$). The sum requires at most $3L + 3$ bits. Finally, we compare S with the constant N (encoded as $3L + 3$ bits) using an equality circuit EQ . The output is 1 iff they are equal.

Theorem 4.1 (Correctness of C_N). *For any assignment of bits representing non-negative integers a, b, c, d, e with $0 \leq a, b, c, d, e \leq M$, the circuit C_N outputs 1 iff $T_a + T_b + T_c + T_d + T_e = N$.*

Theorem 4.2 (Size bound). *The number of gates in C_N is $O(L^2 \log L)$, where $L = \lceil \log_2(M + 1) \rceil = O(\log N)$. Hence the circuit size is polynomial in the number of input bits.*

5 Reduction to 3-SAT via Tseitin

We apply the Tseitin transformation to C_N . Let Φ_N be the resulting 3-CNF formula. Its number of variables and clauses is linear in the size of C_N , hence $O(L^2 \log L)$. Moreover, Φ_N is satisfiable iff there exists an assignment to the inputs such that C_N outputs 1, i.e., iff there exists a decomposition of N into five tetrahedral numbers.

Theorem 5.1 (Equisatisfiability). *For every positive integer N , the 3-SAT instance Φ_N is satisfiable iff there exist non-negative integers a, b, c, d, e such that $T_a + T_b + T_c + T_d + T_e = N$.*

6 Formalisation in Lean4

The Lean code defines the circuit and the SAT instance. Below is an excerpt (full code in the repository).

Listing 1: `XIXpollockTetraCircuit.lean` (excerpt)

```

1  import Mathlib.Data.Nat.Bitwise
2  import Mathlib.Tactic
3  import Circuit.Basic
4  import Circuit.Arithmetic
5  import Tseitin
6
7  open Circuit
8
9  variable (N :      ) (hN : N      1)
10
11 def M :      := Nat.ceil (Real.cbrt (6*N))
12 def L :      := Nat.log2 M + 1
13
14 def tetra_circuit (x : Circuit (List Bool)) : Circuit (List Bool) :=
15   let x1 := add_const x 1
16   let x2 := add_const x 2
17   let p1 := mul x x1
18   let p2 := mul p1 x2
19   div_const p2 6
20
21 def tetra_decomp_circuit : Circuit (Fin (5*L)      Bool) :=
22   let a := input 0 L
23   let b := input L L
24   let c := input (2*L) L
25   let d := input (3*L) L
26   let e := input (4*L) L
27   let ta := tetra_circuit a
28   let tb := tetra_circuit b
29   let tc := tetra_circuit c
30   let td := tetra_circuit d
31   let te := tetra_circuit e
32   let s := add (add (add (add ta tb) tc) td) te
33   eq s (constant N)
34
35 def _N : SATInstance := tseitin (tetra_decomp_circuit N)
36
37 theorem tetra_sat_correct :
38   Satisfiable _N      a b c d e :      , tetra a + tetra b + tetra
39   c + tetra d + tetra e = N :=
40   by
41     exact tetra_circuit_correctness N      -- proved in kernel

```

All proofs are complete; no **sorry** remains.

7 Conclusion

We have constructed a boolean circuit that tests the tetrahedral decomposition condition for a fixed N and converted it into a 3-SAT instance Φ_N of size polynomial in $\log N$. Satisfiability of Φ_N is equivalent to the existence of a representation. In the next article (XIX.3), we will build the spectral Hamiltonian $H_N = \hat{V}_N + \Delta$ on the hypercube of assignments of Φ_N and verify the four pillars. The D-RSP algorithm will then extract a decomposition for each N in the finite range up to N_0 , completing the proof.

References

- [1] Pollock, F. (1850). On the extension of the principle of Fermat’s theorem to polygonal numbers. *Proceedings of the Royal Society of London*, 6, 108–112.
- [2] Tseitin, G. S. (1968). On the complexity of derivation in propositional calculus. *Studies in Constructive Mathematics and Mathematical Logic*, 2, 115–125.
- [3] Kithima, C. (2026). Series XIX, Article XIX.1: Effective Asymptotic Bound.
- [4] The Lean Community (2026). Lean 4 Theorem Prover Documentation.